Mobile Application Development (MAP711S)

Android Libraries

Simon H. Muchinenyika

Android Libraries

Table of Contents

- UI Layer Libraries
  - View Model, Views, Jetpack Compose, View binding,
  - Data Binding Library
  - Paging library
- Data Layer Libraries
  - Repository levels
  - Offline first design
  - DataStore API

Android Libraries

UI Layer Recap

- Purpose:
  - displays application data on the screen.
  - serves as the primary point of user interaction.
- Defining UI State:

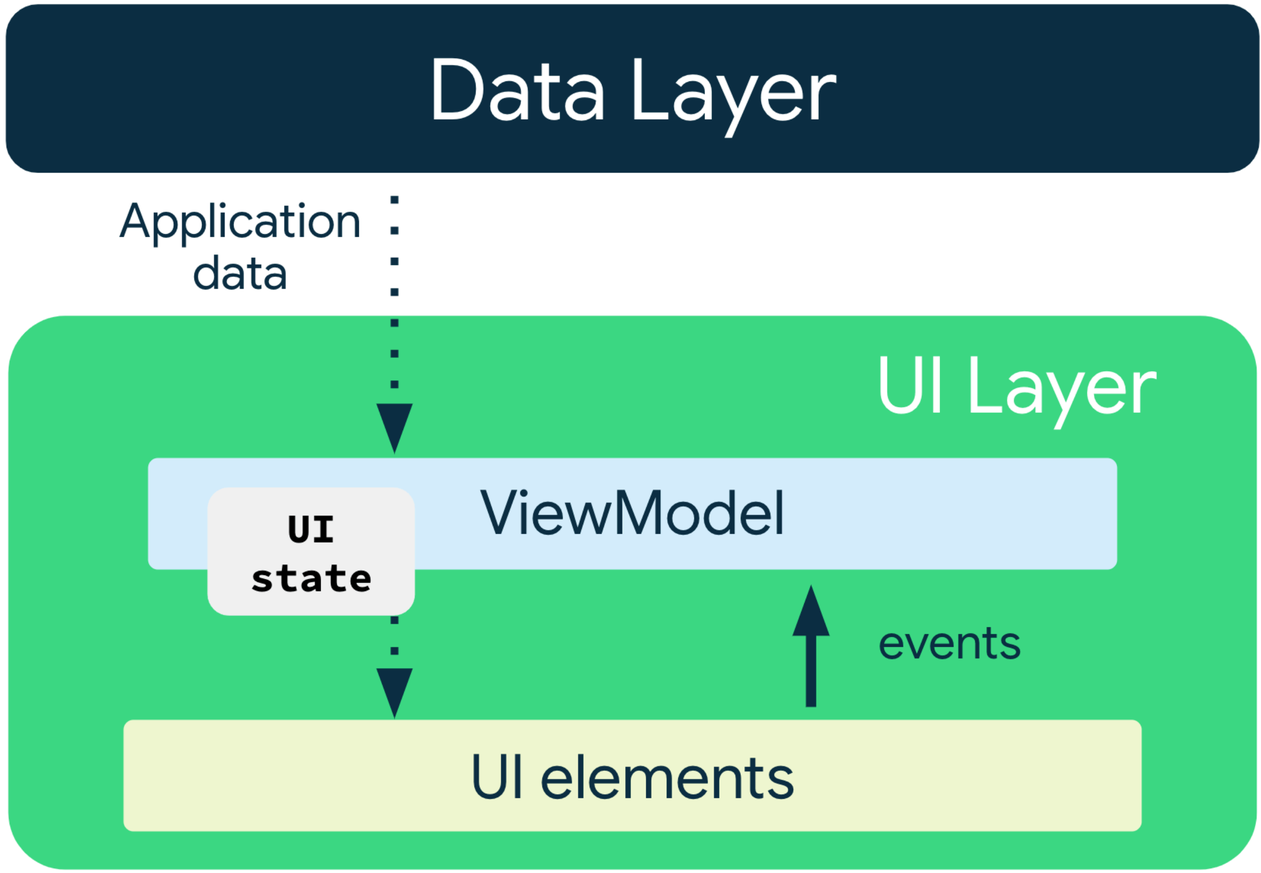
```
data class NewsUiState(
    val isSignedIn: Boolean = false,
    val isPremium: Boolean = false,
    val newsItems: List<NewsItemUiState> = listOf(),
    val userMessages: List<Message> = listOf()
)
```

UI Layer Pipeline:

Language Fundamentals

Unidirectional Data Flow (UDF)

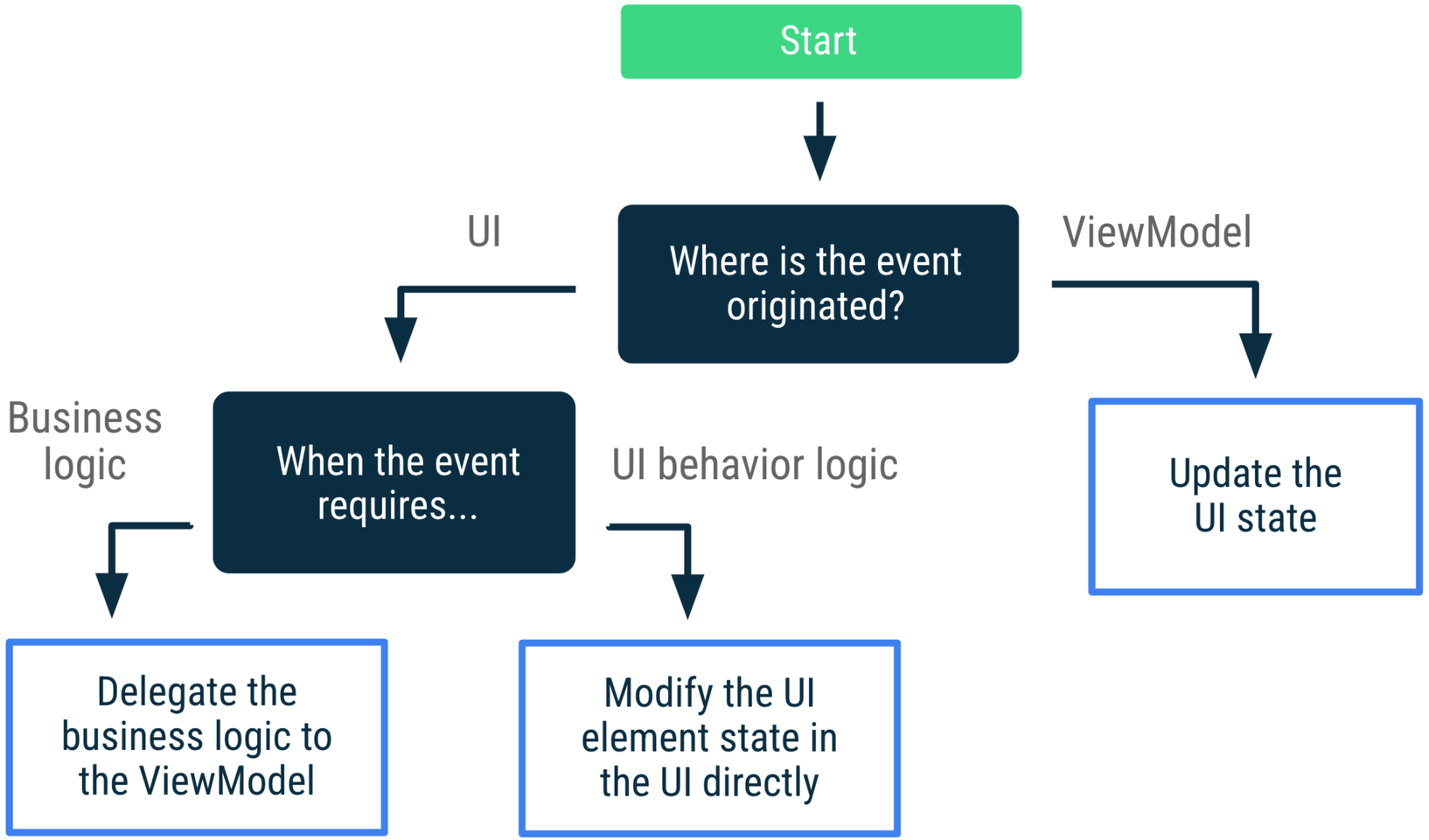
- pattern where the state flows down and the events flow up.
  - ViewModel holds application state.
  - UI notifies ViewModel of user events.
  - ViewModel responds & updates state.
  - Updated state exposed to UI for rendering.
  - Repeated where necessary.



Language Fundamentals

UI Events

- actions handled by the UI or ViewModel.
- business logic had to do with state changes.
- UI had to do with how to display the state changes.



Language Fundamentals

Expose UI State

- Recap: You defined the UI state & managed its production using a UDF.
- Note that using UDF means multiple versions are produced over time, and the resulting state can be considered a stream.
- As such, expose the UI state in an observable data holder like LiveData or StateFlow.
- This make it possible for the UI to respond to state changes without manually pulling data from the ViewModel.

```
class NewsViewModel(...) : ViewModel() {

    val uiState: StateFlow<NewsUiState> = …
}
```

Language Fundamentals

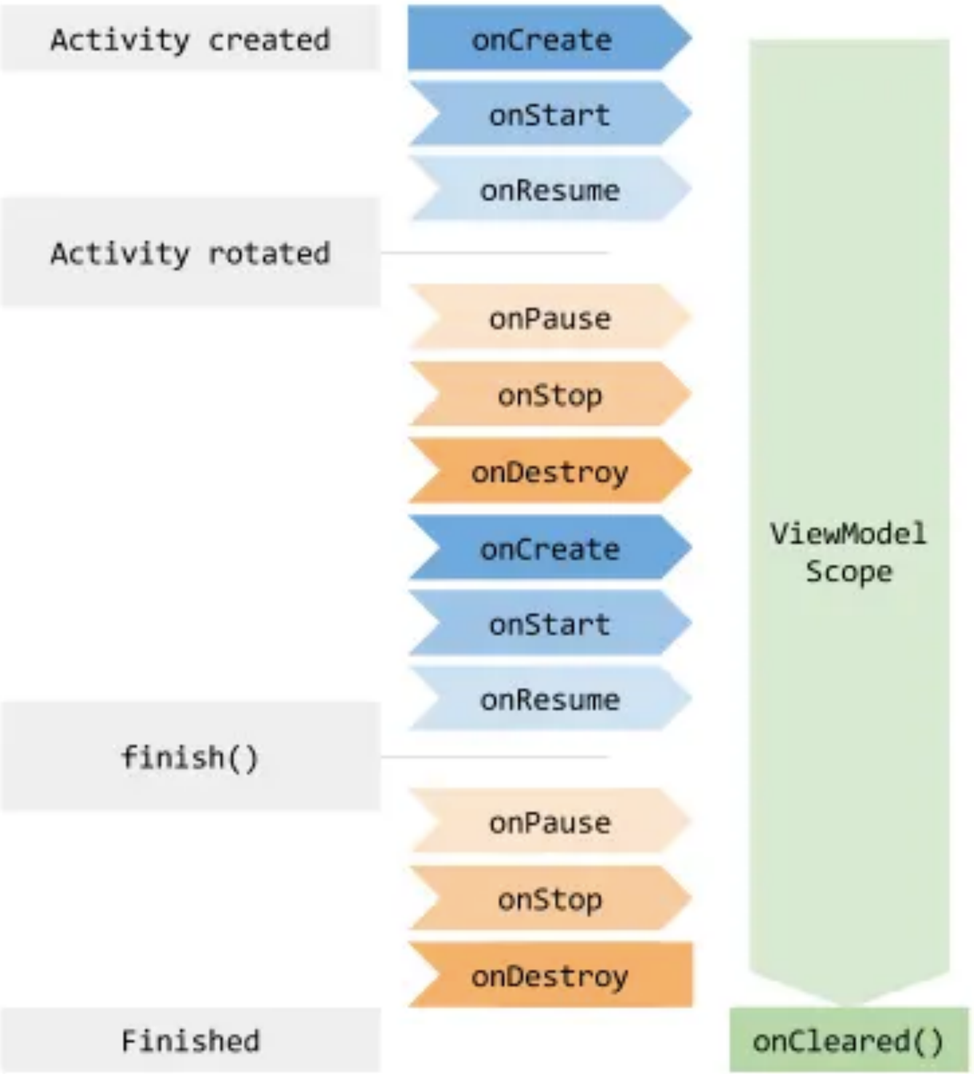
Observer Design Pattern

- bahavioural design pattern that defines a one-to-many relationship.
- A Subject (or Observable) maintains a list of its dependents called Observers.
- The Subject notifies all the Observers of a change in state.
- If you think in terms of publishers, Publishers (Subject) and Subscribers (Observers) form an Observer Design Pattern.
- Implementation: Identify and separate aspects that vary from those that do not.
  - Program an interface & not an implementation.
  - Use composition instead of inheritance.

Language Fundamentals

View Model

- introduced 2017 to hold and manage UI-related data in a life-cycle conscious way.
- Typical Implementation:
  - Separate out your data from your UI controller by creating a class that extends ViewModel.
  - Set up communications between your ViewModel and UI controller.
  - Use your ViewModel in your UI controller.
- ViewModel should not hold a reference to UI controllers.



Language Fundamentals

Consume UI State

- to consume the stream of UiState objects in the UI, use the termina operator for the observable data type you used.
- for LiveData you use the observe( ) method, for Kotlin flows use the collect() method or its variations.
- NB: The UI shouldn't observe the UI state when the view isn't displayed to the user.

```
class NewsActivity : AppCompatActivity() {

    private val viewModel: NewsViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        ...

        lifecycleScope.launch {
            repeatOnLifecycle(Lifecycle.State.STARTED) {
                viewModel.uiState.collect {
                    // Update UI elements
                }
            }
        }
    }
}
```

Language Fundamentals

Threading & Concurrency

- any work performed by the ViewModel should be safe to call from the main thread.
- note that data and domain layers are responsible for moving data to a different thread.
- Kotlin coroutines provide a way of managing concurrent operations, & the Jetpack Architecture Components provide build-in support for them .

Language Fundamentals

View Binding

- makes it easier to write code that interacts with views.

```
android {
    ...
    buildFeatures {
        viewBinding = true
    }
}
```

- once enabled, a binding class is generated for each XML layout in that module.
- The Pascal case of the XML file name is used for the class name, with 'Binding' appended to the end. For example, a file result_proflie.xml becomes ResultsProfileBinding.
- an instance of the binding class contains direct references to all views that have the ID in the corresponding layout.

Android Libraries

Further Study

- Kotlin coroutines: https://developer.android.com/kotlin/coroutines
- Kotlin Flows: https://developer.android.com/kotlin/flow
- ViewModel: https://medium.com/androiddevelopers/viewmodels-a-simple-example-ed5ac416317e

Android Libraries

References

- Android Developer: https://developer.android.com
- Kotlin Flows: https://medium.com/androiddevelopers/a-safer-way-to-collect-flows-from-android-uis-23080b1f8bda
- Observer Design Patterns: https://medium.com/@manishkumar_75473/observer-design-pattern-in-android-the-foundation-of-livedata-23d7b187b1f8
- Headfirst Design Patterns

Android Libraries

Thank You

Questions?

T: +264 61 207 2054

F: +264 61 207 9054

E: smuchinenyika@nust.na